StartupForge AI

Member 1 implementation guide

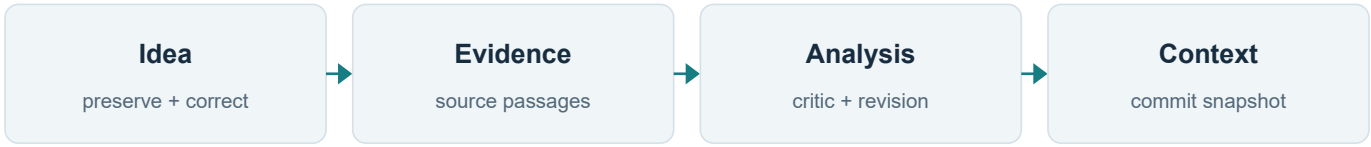
Idea Intelligence | Schema 1.0 | 8 October 2026

Current readiness

The independently runnable module is configured for **offline demo mode**. No model API key has been supplied. Demo responses and debates are scripted; sample sources and statistics are synthetic. Live adapters exist, but live model quality has not been evaluated.

What Member 1 delivers

A user enters a startup idea, corrects its structured fields, retrieves source passages, inspects a reviewed analysis, and opens a debate. The module saves the reviewed findings as an immutable shared context. Statistics appear as exact quoted values with source passages; no charts are generated.



Owner	Responsibility at final assembly
Member 1	Idea extraction, research/retrieval, analysis, critic, one bounded revision, sourced statistics and debates.
Member 2	Pitch deck and poster generation from the committed ProjectContext snapshot.
Member 3	Shared host, projects, contexts, jobs, persistence, access control and report generation. Mounts the feature and supplies final adapters.

Scope and authoritative files

This is the updated Member 1 handoff. The original three-member division remains in `IMPLEMENTATION_PLAN.md`; the historical overall implementation PDF is unchanged. Read the current root `README.md` and `integration/MEMBER1_CONTRACT.md` for the runnable configuration and complete contract. `frontend/README.md` documents mounting.

Feature folders: `backend/app/modules/intelligence` and `frontend/src/modules/intelligence`. The standalone project/job/storage harness is separate in `backend/app/harness`.

Run the standalone application

Installed workspace: one command

From the repository root in PowerShell, start one Uvicorn worker. The production frontend has already been built.

```
.\backend\.venv\Scripts\python.exe -m uvicorn app.main:app `
  --app-dir backend --host 127.0.0.1 --port 8000
```

Open `http://127.0.0.1:8000/` for the application and `http://127.0.0.1:8000/docs` for interactive API documentation. `GET /api/v1/health` reports mode, configured providers and capabilities.

Fresh checkout

Use Python 3.10+ and Node 22.12+. Install backend dependencies and build the frontend:

```
python -m venv backend\.venv
.\backend\.venv\Scripts\python.exe -m pip install -r backend\requirements.txt
cd frontend
npm ci
npm run build
cd ..
.\backend\.venv\Scripts\python.exe -m uvicorn app.main:app `
  --app-dir backend --host 127.0.0.1 --port 8000
```

For frontend development, run `npm run dev` from `frontend` while the backend runs on port 8000. Open `http://127.0.0.1:5173`; Vite proxies `/api` to the backend.

Walk through the product

1. Enter an idea, structure it, and correct the extracted fields. Keep the original input inspectable.
2. Research the idea. Inspect final analysis, excerpts, sourced statistics, critic feedback and initial analysis.
3. Choose You vs. AI with a for/against stance, or watch Pro vs. Skeptic. Let the bounded debate end in a moderator summary.
4. Save the reviewed context and download its JSON for teammates. Reopen a recent project to recover saved context and completed research.

Persistence and restart behavior

SQLite stores projects, analyses, jobs, debates and context versions in `backend/data/intelligence.sqlite3`. Startup marks interrupted jobs failed. The local harness does not resume partially completed jobs or debates.

Configure live models and research

Copy `backend/.env.example` to `backend/.env`, edit settings on the server, and restart the backend. Credentials stay on the backend. Live provider errors become failed jobs; live mode never silently switches to demo data.

OpenAI-compatible service

```
APP_MODE=live
LLM_PROVIDER=openai_compatible
LLM_API_KEY=your-key
LLM_BASE_URL=https://api.openai.com/v1
LLM_MODEL=your-model-id
LLM_RESPONSE_FORMAT=json_schema
SEARCH_PROVIDER=wikipedia
```

The adapter uses Chat Completions with a JSON schema. Set `LLM_RESPONSE_FORMAT=json_object` only if the selected service requires it. The adapter does not automatically retry a billable request in another format.

Keyless local model option: Ollama

```
APP_MODE=live
LLM_PROVIDER=ollama
OLLAMA_BASE_URL=http://127.0.0.1:11434
LLM_MODEL=your-installed-model-name
SEARCH_PROVIDER=wikipedia
```

Ollama must be running and the chosen model must already be installed. This option needs no model API key; it still requires a working local model service.

Choice	Configuration and practical meaning
Wikipedia	`SEARCH_PROVIDER=wikipedia`: keyless background research; not comprehensive commercial market research.
Tavily	`SEARCH_PROVIDER=tavily` and `TAVILY_API_KEY`: broader web retrieval. Review source quality and applicability.
Keyword retrieval	Default transparent matching baseline. It is not semantic embedding retrieval.
FastEmbed	Install `backend/requirements-semantic.txt`; set `RETRIEVAL_PROVIDER=fastembed`. First use may download the embedding model.

Review dates, geography, population and methodology before treating any quoted number as a market estimate. Real accounts have not been called for this implementation, so provider quality still needs evaluation with real ideas.

Reviewed analysis and statistic provenance

The pipeline separates collected documents from individually referenced evidence passages, then generates analysis. A separate critic supplies feedback; at most one model revision is retained alongside the initial and final outputs.

Object	What to preserve and inspect
IdeaSpec	Original input, project name, problem, solution, users, domain, assumptions, constraints and field origins (provided/inferred/unknown).
Source + EvidenceItem	Source ID, URL and retrieval metadata; evidence ID, source ID, exact excerpt and optional locator.
Analysis + critic	Initial output, final output, claim classifications and references; review status ready / ready_with_gaps / blocked, issues and unresolved gaps.
ExecutionInfo	Mode, model provider, search provider, retrieval method and revision count (0 or 1). Keep demo/live provenance visible downstream.

SourcedFigure: exact quotation contract

Every statistic card stores `figure_id`, `display_value`, `metric`, `excerpt`, `evidence_id`, `source_id` and `qualification`. Its source must match the cited evidence source. The excerpt must be a literal substring of the evidence passage, and the displayed value must appear literally inside that excerpt.

Traceable, but still subject to review

A quoted value is not automatically a market size, forecast or generalizable finding. Evidence IDs and exact quotations provide traceability; they do not prove source accuracy, semantic entailment or correct interpretation. All figures in the demo fixture are synthetic.

Boundary validations

- Source, evidence, figure and final claim IDs are unique. Evidence references known sources; final claims reference known evidence.
- A final claim marked supported must include evidence references. Unsupported content is qualified as an assumption or gap.
- Figures appear both in AnalysisResult and its ContextDraft; those figure lists must match. Saving the context carries the figures to teammates.
- Statistics extraction recognizes common digits, currencies, percentages and population units. Coverage is intentionally bounded and does not include every numeric format.

Research motivation

RAG motivates retrieval plus generation; Self-Refine motivates a feedback and revision pass. These design references do not claim that this prototype reproduces the papers' results. A separate critic helps expose gaps but is not an independent factual guarantee.

References: RAG - https://arxiv.org/abs/2005.11401; Self-Refine - https://arxiv.org/abs/2303.17651.

Debates, jobs and safe retries

Mode	Bounded behavior
user_vs_agent	User chooses for or against. The AI takes the opposite stance, opens at round 0, then responds once after each user argument.
agent_vs_agent	Pro then Skeptic alternate per round and run automatically. Users observe the agents.
Completion	API max_rounds: 1-5; current UI: 1-3. Completed debates include a qualitative moderator summary.

Debate invariants

- Pin each debate to its immutable analysis_id and project. Reject cross-project input and analyses with blocked critic results.
- Exclude concurrent user turns. A failed AI reply leaves the prior transcript intact; unknown model citation IDs fail the turn.
- Each selected figure must cite evidence used by that turn. AI debate prose excludes numerical claims; exact quoted cards carry statistics. The conservative prose filter is not a general semantic fact checker.

Debates remain a separate transcript

Saving context does not endorse user arguments, choose a winner, or turn debate claims into reviewed findings. Applying debate feedback to research would require a new review and a new context version.

Generation job protocol

Extraction, analysis, debate creation and user-turn POSTs require `Idempotency-Key` and return **202 JobReceipt**: `job_id`, `status_url`, `operation`, `project_id`. Poll the returned status URL through `queued`, `running`, `succeeded` or `failed`. A 202 acceptance is not success.

- Extraction results are in Job.result. Analysis/debate jobs expose their resource through Job.result_url. Only completed analysis jobs feed downstream generation.
- A running debate may publish its transcript URL and partial turns. Poll the job to learn whether generation completed or failed.
- Replay the same key and payload to recover the same job. Changed payload returns 409. Keys are scoped by project and operation, plus debate ID for user turns.

Errors and context commits

```
{error:{code,message,details,retryable,request_id}}
```

Failed jobs contain that same inner error object. `X-Request-ID` is echoed for diagnostics. Context commits use optimistic `expected_latest_version` comparison, rather than generation job idempotency; another saved version causes a 409 conflict.

Required HTTP endpoints - all 15

Every route below includes `/api/v1`. M1 = feature owner; M3 = shared host owner, with a local harness supplied for independent development.

Verb	Full route	Request / result / owner
GET	`/api/v1/health`	No body. Mode, providers, capabilities. Shared host; local implementation.
POST	`/api/v1/projects`	display_name, idea_text. 201 project metadata. M3.
GET	`/api/v1/projects?limit=20`	Optional limit: 1-100. {items, next_cursor}. M3.
GET	`/api/v1/projects/{project_id}`	No body. Project metadata with latest_context_version. M3.
POST	`/api/v1/intelligence/extractions`	project_id, idea_text. 202 JobReceipt; Job.result is IdeaSpec. M1.
POST	`/api/v1/intelligence/analyses`	project_id, idea, research_options. 202 JobReceipt; analysis result_url. M1.
GET	`/api/v1/intelligence/analyses/{analysis_id}`	AnalysisResult: context_draft, figures and execution details. M1.
POST	`/api/v1/intelligence/debates`	project_id, analysis_id, mode, user_stance, max_rounds. 202 receipt. M1.
GET	`/api/v1/intelligence/debates/{debate_id}`	DebateSession: turns, citations, figures and summary. M1.
POST	`/api/v1/intelligence/debates/{debate_id}/turns`	message. 202 JobReceipt. User debate only. M1.
GET	`/api/v1/jobs/{job_id}`	Job: status, progress, result, result_url or structured error. M3.
GET	`/api/v1/projects/{project_id}/jobs`	{jobs, next_cursor}; latest first. M3.
POST	`/api/v1/projects/{project_id}/contexts`	expected_latest_version, context. 201 immutable context; conflict 409. M3.
GET	`/api/v1/projects/{project_id}/context`	Latest committed ProjectContext. M3.
GET	`/api/v1/projects/{project_id}/contexts/{version}`	Exact immutable ProjectContext. M3.

Required header: Idempotency-Key on the four feature generation POSTs (extractions, analyses, debates and debate turns). For analysis, research_options includes max_sources (1-8), language (default en) and optional region.

The original overall plan also defines teammate output APIs. They are consumers of the saved snapshot and are outside this Member 1 route catalog.

Shared snapshot and final assembly

The committed **ProjectContext** is the integration boundary. It contains the complete ContextDraft plus server-assigned `project_id`, `context_version`, `created_at` (UTC) and `content_hash`. Treat each saved version as immutable.

Use the committed JSON and schemas independently

```
integration/fixtures/project_context.demo.json
integration/fixtures/analysis_result.demo.json
integration/schemas/ProjectContext.schema.json
integration/schemas/ContextDraft.schema.json
integration/schemas/IdeaSpec.schema.json
integration/schemas/AnalysisResult.schema.json
integration/schemas/DebateRequest.schema.json
integration/schemas/DebateSession.schema.json
integration/schemas/JobReceipt.schema.json
integration/schemas/Job.schema.json
```

Member 2 and Member 3 can build and test against these fixtures without running Member 1. Freeze schemas by team agreement. If the agreed boundary changes, regenerate artifacts with `backend/scripts/export_contracts.py` and update consumers and fixtures together.

Mount once at final assembly

1. Member 3 supplies IntelligenceRepository and JobRunner adapters from interfaces.py. Keep the shared project/context/job endpoints; replace the local app/harness rather than mounting it twice.
2. Build IntelligenceService with selected model, search and retrieval adapters. Build DebateService with the same repository and model, then include build_router(IntelligenceDependencies(...)) in the shared FastAPI app.
3. Mount the exported IntelligenceWorkspace React component with apiBase="/api/v1". Its optional onContextSaved callback receives the complete committed snapshot.
4. Extract and correct the idea, finish analysis, inspect context_draft, then commit it using the current expected_latest_version. Use the returned immutable snapshot, including its identity metadata.
5. Pass the full snapshot to Member 2 pitch/poster and Member 3 report generation APIs. Consume final_analysis, sources, evidence, figures and unresolved gaps. initial_analysis is audit material and may intentionally contain unsupported claims.

```
import { IntelligenceWorkspace } from "./modules/intelligence"

<IntelligenceWorkspace
  apiBase="/api/v1"
  onContextSaved={(context) => workspace.openContext(context)}
/>
```

Downstream routes from the original plan: /api/v1/studio/pitch-decks, /api/v1/studio/posters and /api/v1/reports/generations. Use the templates and options defined by the owning module.

Adapter responsibilities and acceptance

Boundary source: `backend/app/modules/intelligence/interfaces.py`. The feature router is in `router.py`; service construction is in `pipeline.py`. Member 3 owns authentication and project access checks in the shared host.

Adapter	Required responsibility
IntelligenceRepository	get_project; save_analysis / get_analysis; save_debate / get_debate; claim_debate / release_debate. Preserve typed results and atomically exclude concurrent turns.
JobRunner	submit operation, project, scoped key, payload, async work, request_id and optional scope_suffix. Persist idempotency; supply progress callbacks and shared job endpoints.
Result publication	Implement publish_result_url(job_id, result_url) so partial debate transcripts can become discoverable while generation runs. Work returns JobOutcome with result and/or result_url.
Production runtime	Use durable shared queues and recoverable owner-safe leases across workers. The local SQLite/in-process prototype uses one Uvicorn worker and cannot provide these guarantees.

Verify the module

```
cd backend
.\.venv\Scripts\python.exe -m pytest -q
cd ..\frontend
npm run build
```

The current backend suite passed 99 tests, and the production frontend build passed. Tests cover mocked model/search contracts, pipeline review and citations, exact figure quotations, idempotent jobs, immutable contexts, debate stance and round limits, concurrency and provider failures. The browser workflow is checked separately; these checks do not evaluate real model quality. A real Wikipedia network smoke check returned HTTP 403 in this environment. Keyless configuration therefore does not establish working network access here; live provider contract checks used mocked HTTP.

Final shared-project acceptance

- Commit context v1, generate pitch/poster/report outputs from v1, commit v2, and confirm the older outputs still identify v1.
- Replay same-key requests; reject changed payloads. Exercise provider failure, two simultaneous debate turns and a context version conflict.
- Check extraction corrections, reviewed analysis, both debate modes, citation/figure cards, JSON download and project reopen against the real backend.
- Keep fixture/synthetic labels in demo mode. With live configuration, evaluate source quality, relevance, unsupported assumptions and numerical claims using real startup ideas.

Reference trail

Current local sources: `README.md`, `integration/MEMBER1_CONTRACT.md`, `frontend/README.md`, module contracts and interfaces. Official provider references linked in the README:

OpenAI structured outputs: developers.openai.com/api/docs/guides/structured-outputs
Ollama chat API: docs.ollama.com/api/chat
Tavily search API: docs.tavily.com/documentation/api-reference/endpoint/search
FastEmbed: qdrant.github.io/fastembed/